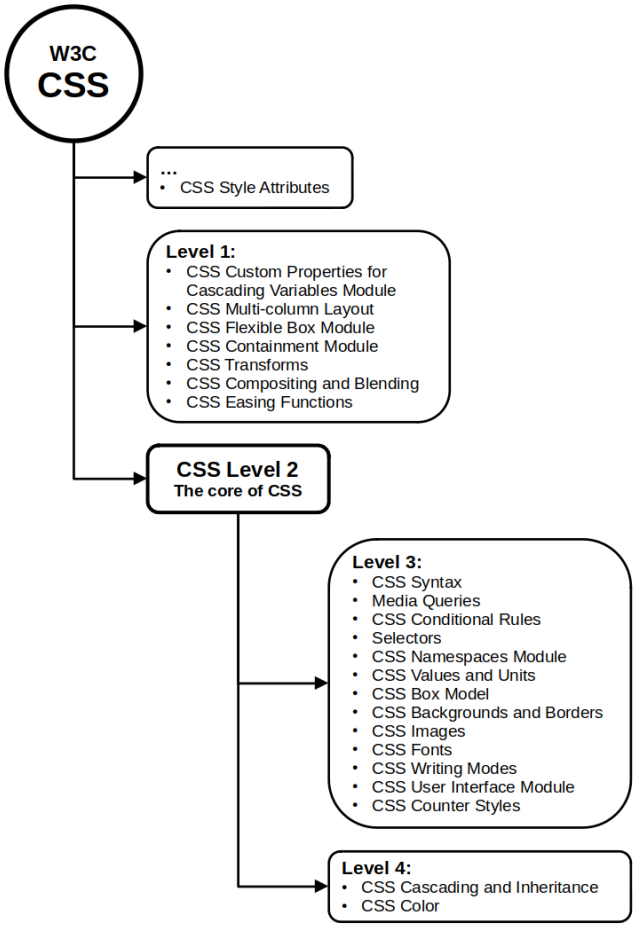
Plan

1. Introduction
2. Syntaxe des feuilles de style
3. Mise en forme du texte
4. Mise en forme de la page
5. Site web reactif (responsive)

Definition (CSS)

CSS (*Cascading Style Sheets* = feuilles de styles en cascade) est un langage permettant la **description** de la représentation graphique des pages HTML.

- Le CSS fournit des informations complémentaires à HTML.
- Le standard est développé par W3C via un un groupe de travail incluant les acteurs majeurs du web.
- Il est développé par niveaux.



Principe : séparation du fond et de la forme

Même idée que les styles des traitements de texte

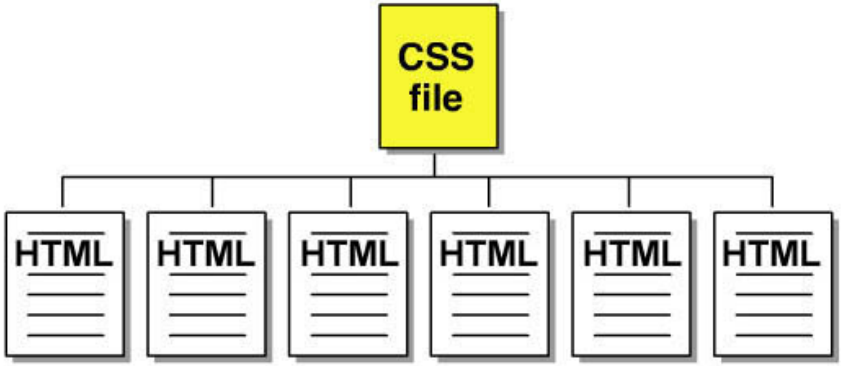
- Distinction entre la structuration du texte (HTML) et les propriétés typographiques (CSS)que l'on applique au texte.
- Les styles permettent d'appliquer certaines propriétés à des balises HTML.

Bonne pratique

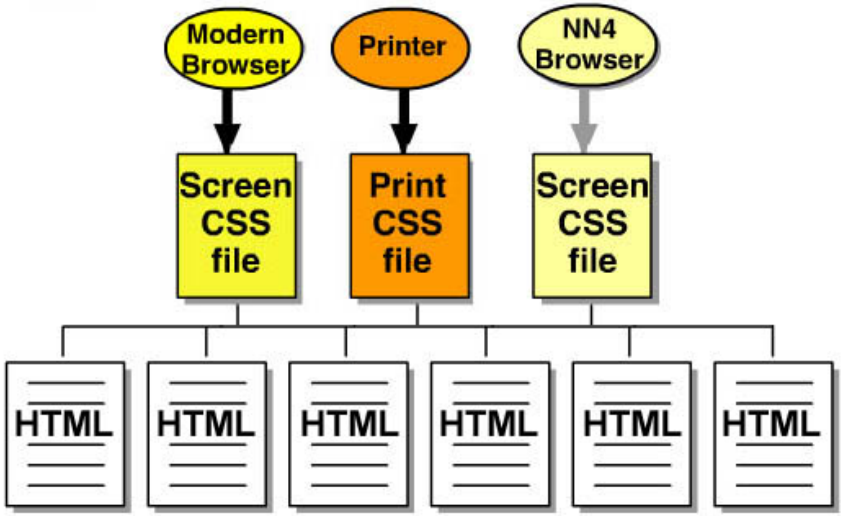
Une page web bien conçue ne devrait contenir aucune information de style dans le fichier html.

Principe : séparation du fond et de la forme

- Plusieurs documents peuvent partager les mêmes instructions typographiques.
- Homogénéité de typographie et de présentation des pages auxquelles s'appliquent les styles.

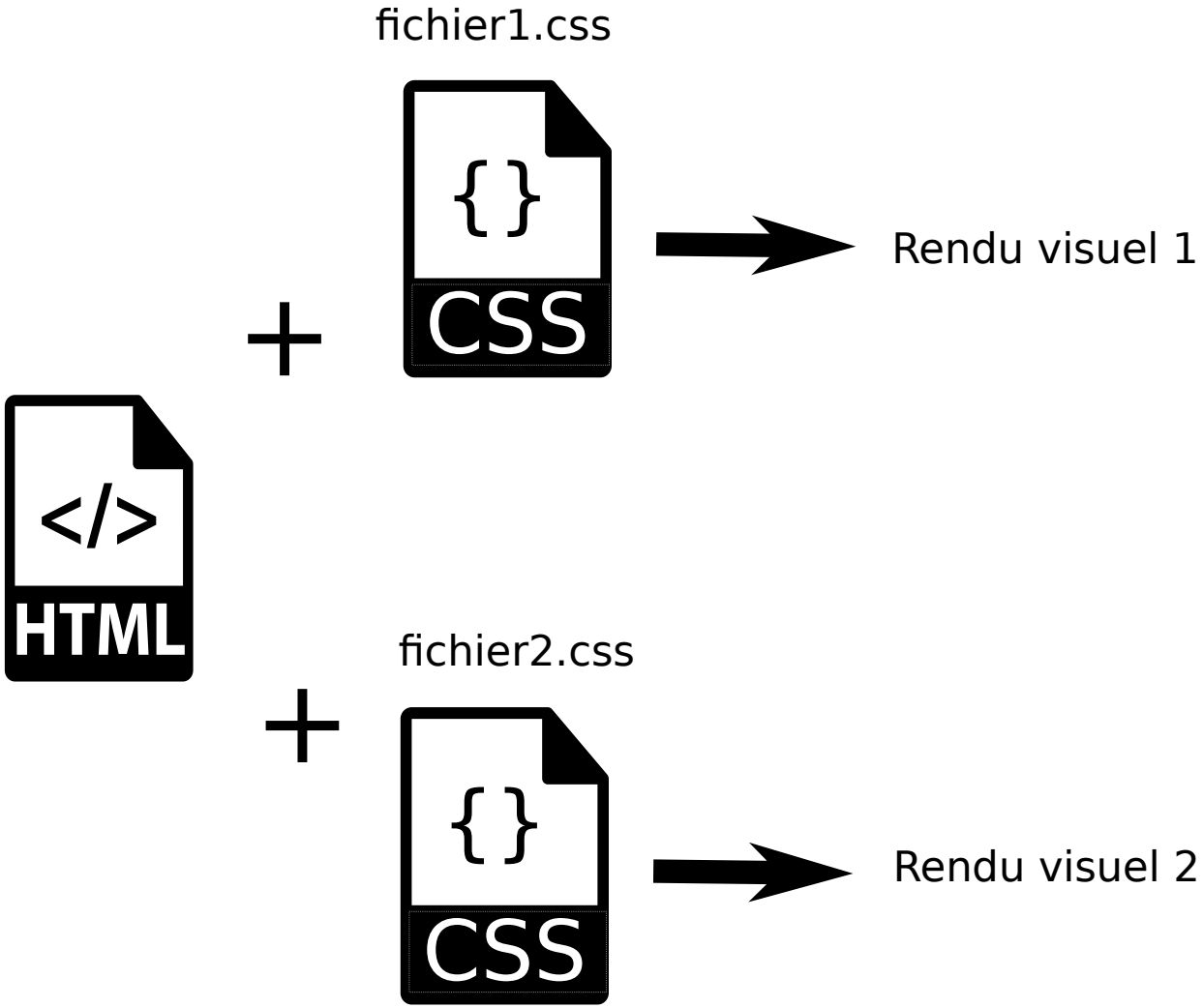


- On peut définir des styles adaptés au support (type et taille d'écran, impression papier).



Principe : séparation du fond et de la forme

L'apparence des documents peut facilement être modifiés en utilisant les styles définis



Séparation du fond et de la forme : exemple

On considère la page web suivante

```html
<html lang="fr">
<head>
   <meta charset="UTF-8">
   <title>La Vie d'Alan Turing</title>
</head>
<body>
   <nav>
      <a href="#">Accueil</a>
      <a href="#">Biographie</a>
      <a href="#">Contributions</a>
      <a href="#">Héritage</a>
   </nav>
   <div class="content">
      <h1>La Vie d'Alan Turing</h1>
      <section>
         <h2>Qui était Alan Turing ?</h2>
         <img width="200px" src="alan-turing.jpeg" alt="Photo␣d'A.␣Turing" />
         <p> Alan Turing (1912-1954) était.....militaire du 20e siècle.</p>
      </section>
      <section>
         <h2>Ses Contributions</h2>
         <p>Alan Turing a marqué un.....plusieurs batailles stratégiques.</p>
      </section>
      <section>
         <h2>L'Héritage de Turing</h2>
         <p>L'héritage d'Alan Turing ..... erreur historique.</p>
      </section>
   </div>
</body>
</html>
```

Résultat dans firefox sans style

La Vie d'Alan Turing — Mozilla Firefox

La Vie d'Alan Turing

file:///media/christophe/EMTEC B250/Ensei

Accueil Biographie Contributions Héritage

La Vie d'Alan Turing

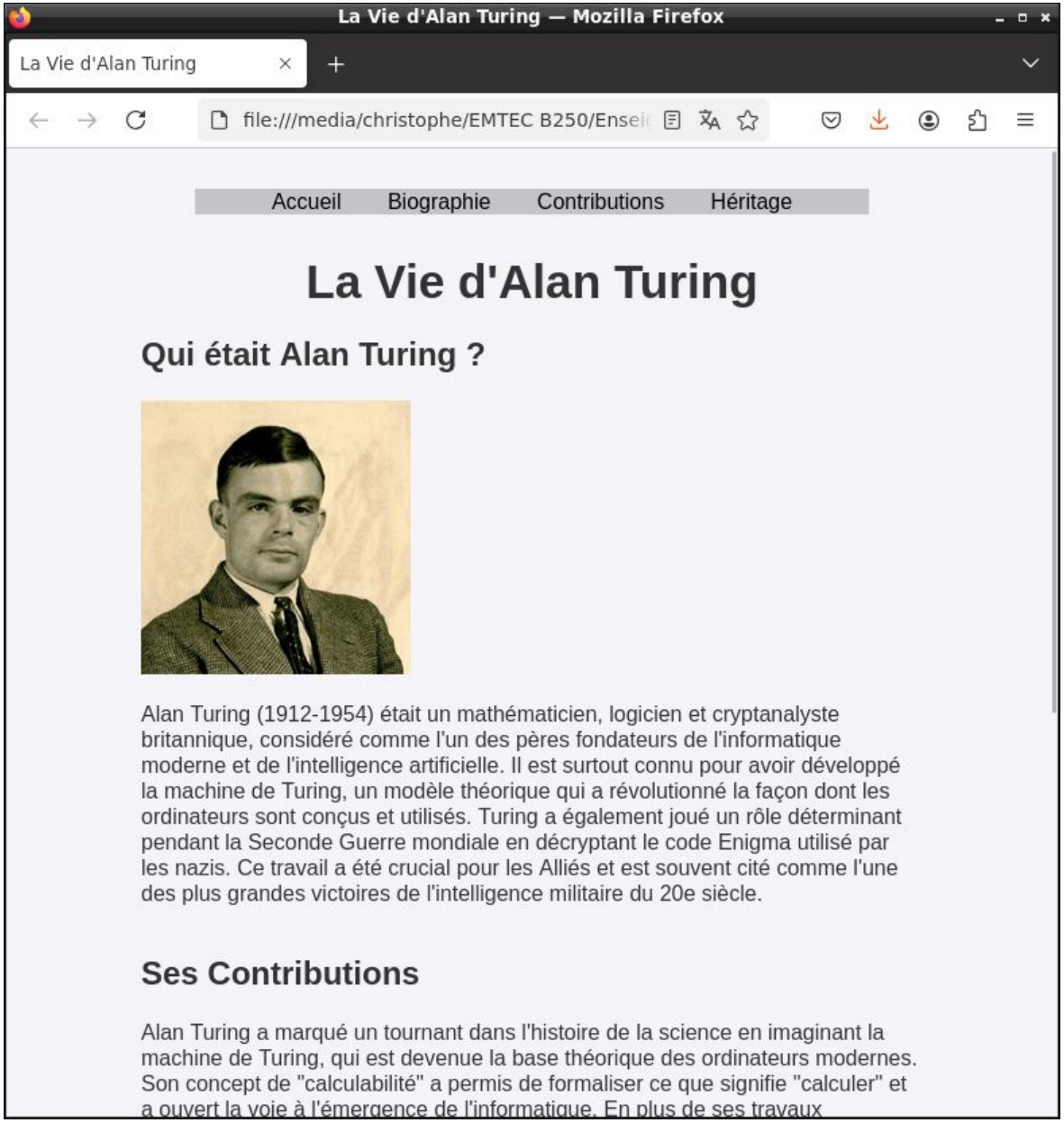
Qui était Alan Turing ?

Alan Turing (1912-1954) était un mathématicien, logicien et cryptanalyste britannique, considéré comme l'un des pères fondateurs de l'informatique moderne et de l'intelligence artificielle. Il est surtout connu pour avoir développé la machine de Turing, un modèle théorique qui a révolutionné la façon dont les ordinateurs sont conçus et utilisés. Turing a également joué un rôle déterminant pendant la Seconde Guerre mondiale en décryptant le code Enigma utilisé par les nazis. Ce travail a été crucial pour les Alliés et est souvent cité comme l'une des plus grandes victoires de l'intelligence militaire du 20e siècle.

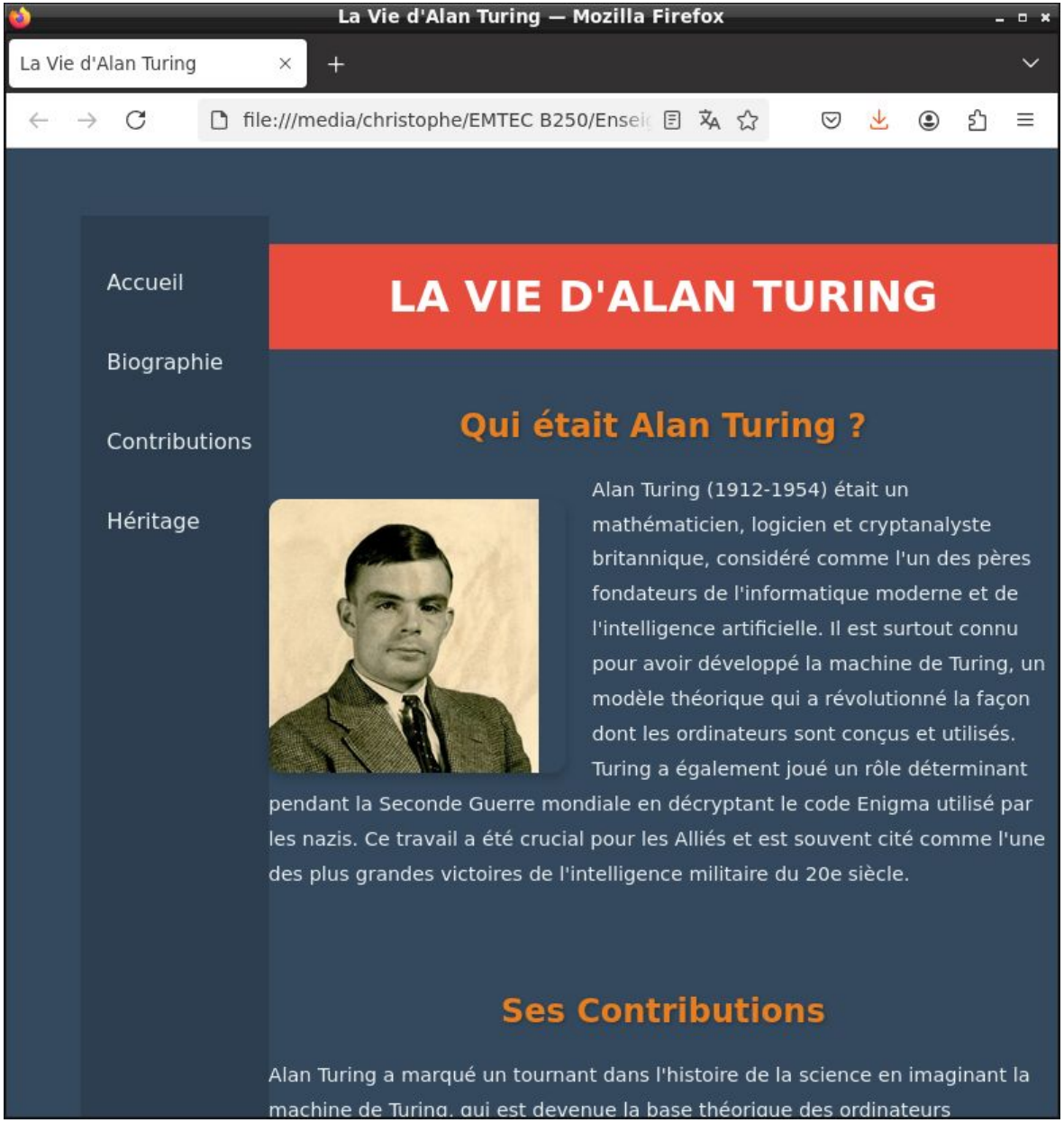
Ses Contributions

Alan Turing a marqué un tournant dans l'histoire de la science en imaginant la machine de Turing, qui est devenue la base théorique des ordinateurs modernes. Son concept de "calculabilité" a permis de formaliser ce que signifie "calculer" et a ouvert la voie à l'émergence de l'informatique. En plus de ses travaux théoriques, Turing a participé à l'effort de guerre en travaillant à Bletchley Park, où il a conçu une machine capable de déchiffrer les messages codés par la machine Enigma. Cela a permis aux Alliés de prévoir les mouvements de l'ennemi et de remporter plusieurs batailles stratégiques.

Résultat dans firefox avec turing-style1.css



Résultat dans firefox avec turing-style2.css



Plan

1. Introduction
2. Syntaxe des feuilles de style
3. Mise en forme du texte
4. Mise en forme de la page
5. Site web reactif (responsive)

Exemple simple

Page HTML :

```
1  <body>
2    <h1> Happy Together </h1>
3    <p> Film de Wong Kar-Wai
4        réalisé en 1997 </p>
5  </body>
```

Fichier CSS:

```
1  body {
2    color:#000080;
3  }
4
5  h1 {
6    font-weight:bold;
7    font-style:italic;
8    color:red;
9    font-size:24px;
10 }
```

Voici le rendu obtenu:

Où définir le style (1/3)

- Méthode 1. Balise `<style>`. Un style peut etre défini dans l'entete (`head`)avec la balise `style`

```
1  <head>
2    <style type="text/css">
3    <!-- On definit le style ici -->
4    </style>
5  </head>
```

- Méthode 2. Dans un fichier `.css`, dans l'entete on rajoute

```
1  <link rel="stylesheet" type="text/css"
2                         ref="style.css">
```

- Méthode 3. A l'intérieur d'une balise avec l'attribut `STYLE`

```
1  <balise style="definition␣du␣style" ... >
2  <!-- Zone active de balise et du style -->
3   </balise>
```

Bonne pratique

Privilégier le style défini dans un fichier (Méthode 2)

Où définir le style (2/3)

- A éviter fortement :

```
1  <h1 style="color:red"> Happy Together </h1>
```

- Acceptable mais à éviter:

```
1  <head>
2   ...
3   <style type="text/CSS">
4   h1 { color:red;}
5  </style>
6  ...
7  </head>
8  <body>
9    <h1> Happy Together </h1>
10 </body>
```

Où définir le style (3/3)

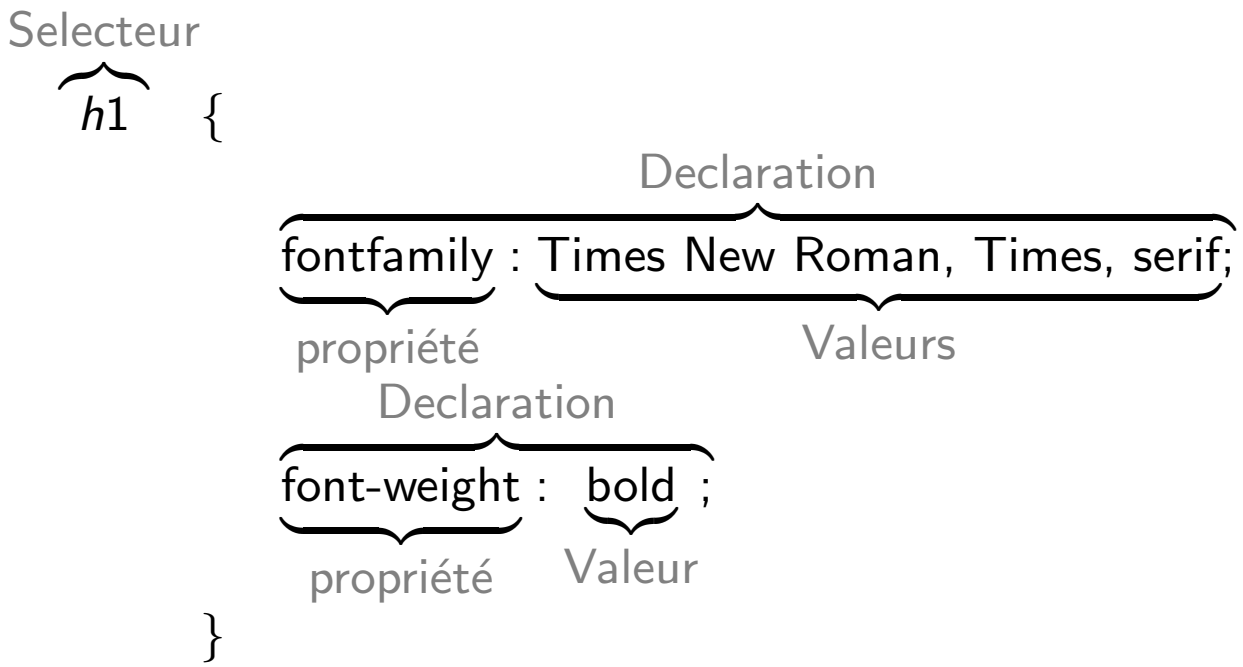
L'approche à privilégier

```
1  <head>
2   ...
3   <link rel="stylelesheet" type=''text/css"␣href="feuille-style.css"␣/>
4  ...
5  </head>
6  <body>
7  ␣␣<h1>␣Happy␣Together␣</h1>
8  </body>
9  ␣␣
```

où `feuille-style.css` est un fichier .css contenant `h1  color:red;`

Syntaxe générale d'une feuille CSS

- Le contenu d'un fichier css ne contient pas d'entête ni pieds
- Il consiste en une série de définition du type



- Le css est insensible aux espaces et retours à la ligne (mais indentez vos fichiers!)

Les commentaires avec /* */

```
1    body {
2      color:#000080; /* la couleur */
3     /* text-decoration: line-through ; */
4  }
```

Syntaxe générale d'une feuille CSS

- Le sélecteur indique à quelles balises s'applique la déclaration de style.
- Il peut y avoir plusieurs sélecteurs pour une déclaration de style.

```
1   h1 {
2       font-weight:bold;
3       font-style:italic;
4       color:red;
5       font-size:24px;
6     }
7   h4, h5, h6 {
8       font-weight:normal;
9       font-style:normal;
10      color:black;
11      font-size:16px;
12    }
```

Syntaxe : identifiant et classes

Identifiants : identifie une balise (de manière unique!) pour appliquer un style individuelement:

- Syntaxe HTML `<h1 id="titre">`
- Remarque : ce sera la seule avec cet id dans la page.
- Syntaxe CSS: `h1 #titre { ... }`

Classes : identifie un groupe de balise

- Syntaxe HTML `<h1 class="titre">`
- Remarque : il peut y avoir plusieurs balises avec `class="titre"`
- Syntaxe CSS: `h1 .titre { ... }`
- on peu attribuer plusieurs classes à une balise:

```
1     <h1 class="classe1 classe2">
```

Syntaxe : identifiant et classes

- Selecteur universel * s'applique à toutes les balises:

```
1    * { font-weight:bold;}
```

- Selecteur d'identifiant s'applique une seule balise identifié par son `id`:
  - Le HTML

```
1    <h1 class="classe1 classe2">
```

  - Le CSS

```
1     h1 #titre { font-size : smaller ; }
```

- Selecteur de classe : s'applique à toutes les balises appartenant à la même classe :
  - Le HTML

```
1    <h2 class="titres">
```

  - Le CSS

```
1     h2 .titre { font-size : smaller ; }
```

On peut attribuer plusieurs classes à une balise:

```
1    <h1 class="classe1 classe2">
```

Exemple id/class

```html
<html>
  <head>
    <style type="text/css" >
      h1 {background-color:green}
      .red {color:red}
      .blue {background-color:lightblue
            }
      #paragraphe {background:lightgrey
            ;color:red;}
      </style>
  </head>
  <body>
  <h1 class="red"> Alan Turing </h1>
  <div>
    <p class="red␣blue">
      Alan Turing (1912-1954) était
            ........ conçus et utilisés
    </p>

    <p class="blue"> Turing a
          .........  du 20e siècle.</p>
    </p>
  <p id="paragraphe"> L'héritage d'Alan
          ......... des débats
        contemporains.
    </p>
    </div>
  </body>
  </html>
```

Quelques autres sétecteurs

- Descendants directs : descendants directs d'un élément. Par exemple pour un <p> dans un :

```
1     div > p { font-family : serif; }
```

- Descendants indirects : descendants directs ou indirects d'un élément

```
1     div span.filmimportant  { font-weight : bold; }
```

- Lien visités :

```
1     :link { text-decoration : underline; }
```

- Élément survolé par la souris :

```
1     h2 .titre { font-size : smaller ; }
```

Héritage

Les propriétés CSS **héritent** des propriétés parentes.

- Héritage par propriété sémantiquement parente :
  - `h2` hérite de `body`
  - `h2` hérite de `*`
- Héritage par raffinement de propriété :
  - `h2#meilleurfilm` hérite de `h2`
  - `h2.bonfilm` hérite de `h2` (mais pas de `h2#meilleurfilm`)

Exemple :

```
1        h2 {background-color:lightblue
             ;}
2        h2#vieuxfilm { color:red;}
3        h2.filmrecent {font-weight:
             italic;}
```

Cascade

Si plusieurs règles s'appliquent à un élément pour la même propriété, les mécanismes suivants sont appliqués :

- Si plusieurs règles sont identiques, la dernière est utilisée
- Si une règle est plus spécifique qu'une autre (i.e., plus proche dans le DOM), la plus spécifique est utilisée:
  - `<div>` est plus spécifique que `*`.
  - div#palmedor est plus spécifique que div
  - p.chefdoeuvre est plus spécifique que .chefdoeuvre.
- On peut augmenter la priorité d'une règle avec le mot-clé !important. Alors cette règle qui est appliquée en premier.

Bonne pratique:

L'utilisation de !important est considéré comme une mauvaise pratique.

Plan

1. Introduction
2. Syntaxe des feuilles de style
3. Mise en forme du texte
4. Mise en forme de la page
5. Site web reactif (responsive)

Propriétés : styles

- Propriété font-size : taille de police
  - Valeurs prédéfinies absolues : xx-small, x-small, medium, large, x-large, xx-large
  - Valeurs prédéfinies relatives : smaller, larger
  - Valeur numérique : x unités, où unité $\in \{pt, px, em, \%\}$
- Propriété font-weight : graisse de la police
  - Valeurs prédéfinies : lighter, normal, bold, bolder
  - Valeurs numérique : $x \in [0; 1000]$ (1000 étant le plus gras)
- Propriété font-style : inclinaison de la police
  - Valeurs prédéfinies : normal, italic, oblique
- Propriété text-decoration : décoration (soulignage, surlignage)
  - Valeurs prédéfinies : underline, overline, line-through, none.

Propriétés : police

- Propriété font-family : famille de police de caractères
  - Grandes familles : serif (police avec empattement), sans-serif (linéale), monospace (police à chasse fixe).
  - Nom de police : Arial, Courier New, Courier, Helvetica, Times New Roman, Times, et bien d'autres
- Exemple :

vif zéphyr	vif zéphyr	vif zéphyr
serif	sans-serif	monospace
avec empattement	linéale	chasse fixe

Bonnes pratiques typographiques

- Les titres sont souvent en gras et le corps en normal.
- Il convient d'éviter de mélanger outre mesure les familles de polices dans un même document.

Propriétés : capitales et petites capitales (1/2)

- Propriété text-transform : police en capitales / minuscules. Valeurs possibles :
  - capitalize : Première Lettre De Chaque Mot
  - uppercase : CAPITALES
  - lowercase : minuscules
  - none : normal
- Propriété font-variant : pour les Petites Capitales. Valeurs :
  - normal : Un cours de Prog Web à l'UPVD
  - small-caps : Un cours de prog web à l'UPVD

Propriétés : capitales et petites capitales (1/2)

Exemple :

```
1  <p> Le président de l'<span style ="text-transform:uppercase"> upvd </span>
       est
2  Yvan <span style="font-variant:small-caps"> Auguet </span >. </p>
```

Affiche: "Le président de l'UPVD est Yvan AUGUET".

Bonnes pratiques typographiques

En HTML, On ne doit pas mettre directement des majuscules mais utiliser les attributs CSS corresondants. (Sauf les majuscules "normale" de début de phrase).

Bonnes pratiques typographiques

- L'usage des majuscules en début de mot est *rare*en français.
- L'usage de mots entièrement en majustucles est quasi-absent dans les textes français (utiliser plutôt les petites capitales).

Couleurs

Propriété color : couleur de la police

- Couleurs prédéfinies : red, blue, green, orange, etc.
  - Voir par exemple `https://web-color.aliasdmc.fr/`
- Valeurs utilisateur :
  - Hexadécimal : #RRVVBB
  - RVB : rgb(r, g, b) avec valeurs de 0 à 255
  - RVB : rgb(r%, g%, b%) avec valeurs de 0 à 100
  - RVB + transparence : rgba(r, g, b, a) avec valeurs de 0 à 255, et transparence (a) entre 0 et 1
  - HSL (*hue + saturation + lightness, teinte + saturation + luminosité)* : hsl et hsla
- Opacité : opacity (valeur entre 0 et 1), alternative à la transparence

```
1  div # palmedor > span . actrice :hover {
2  background-color:hsl(120 , 30% , 50%);
3  color:hsl(0 , 100% , 50%);
4  border-color:hsla(0 , 100% , 25 , 0.5%);
5  opacity:0.8;
6  }
```

Bordure

Les propriétés associées à la bordure sont:

- border-style $\in$ `solid,dashed,dotted,double,...`
- border-width largeur de la bordure
- border-color couleur de la bordure

```
1   <head>
2     <style type="text/css">
3       h1 {border-width:thick;}
4       h1.dotted {border-style:dotted;}
5       h1.dashed{ border-style:dashed;}
6       h1.solid{ border-style:solid;}
7       h1.double{border-style:double;}
8     </style>
9   </head>
10  <body>
11    <h1 class="dotted"> titre dotted </h1>
12    <h1 class="dashed"> titre dashed </h1>
13    <h1 class="solid"> titre solid </h1>
14    <h1 class="double"> titre double </h1>
15  </body>
```

Plan

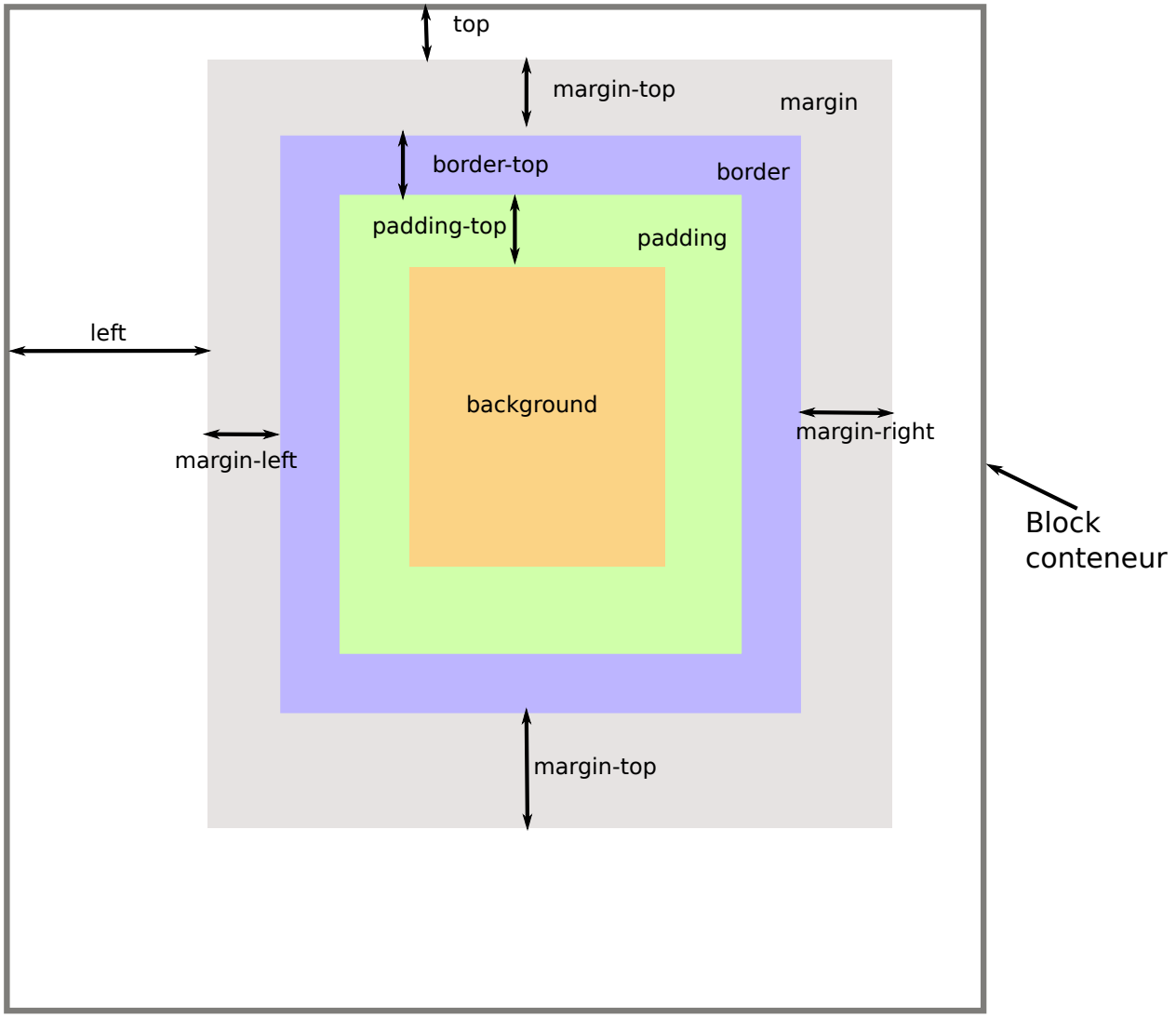
1. Introduction
2. Syntaxe des feuilles de style
3. Mise en forme du texte
4. Mise en forme de la page
5. Site web reactif (responsive)

Positionnement d'une balise de type block

On utilise les propriétés `marging,padding` et `border`

- marging (ou individuellement margin-top,margin-left ...) donne l'espace entre le bloc contenant et le bloc considéré.
- border (ou individuellement border-top, ...) décrit la bordure entourant le bloc.
- padding (ou individuellement padding-top, ...) décrit l'espace entre le contenu de la balise et la bordure.
- top,left,right,bottom donne l'écart entre le block conteneur et le bloc de la balise considérée.
- z-index qui permet si des éléments se chevauche de savoir celui qui est "au dessus".
- On peut spéficier la taille du bloc avec width,height. Dans ce cas les marges bottom et right n'ont pas d'effet.

Marge,Espace intérieur



Positionnement relative, fixe, absolute

On utilise la propriété positionnement avec les valeurs suivantes:

- static est positionné par rapport à l'ordre normal du document (left,top,right,bottom et z-index n'auront pas d'effet). Valeur par défaut.
- relative est positionné par rapport à l'ordre normal du document, les valeurs de left,top,right,bottom auront un effet de décaler la position par rapport au bloc conteneur.
- absolute écart fixe par rapport au bloc conteneur (si le conteneur est déclaré en relative).
- fixed écart fixe par rapport à la fenêtre du navigateur (les valeurs left et top positionne l'élément dans la fenêtre).
- sticky fixe l'élément lorsqu'il arrive en haut de la fenêtre (au décalage top).

Notez que absolute et fixed sort l'élément du flot d'élément (les élément qui suivent ne tiennent pas compte de cet élément pour être positionné).

Exemple

Voici un code avec un div conteneur et trois div inclus (rouge,bleu et vert) de hateur et largeur fixées

```
<head>
  <style tyle="text/css">
    div{
        color:white;
        width:70px;
        height:70px;
  margin:20px;
  display:block;

    }
    div.conteneur{ margin:30px; position: relative;}
    div.bleu {background-color:blue; }
    div.red{
        background-color:red;
        top:20px; left:20px; position: absolute;
    }
    div.green{background-color:green;  }
  </style>
  </head>
<body>
  <div class="conteneur">
    <div class="bleu">div bleu  </div>
    <div class="red"> div rouge </div>
    <div class="green"> div vert </div>
    </div>
</body>
```

Exemple : on fait varier position du div bleu

Static : let/top n'ont pas d'effet

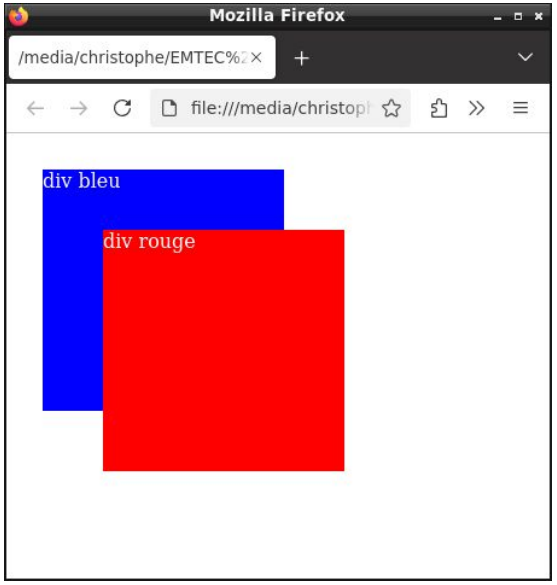
Fixed : positionné avec left/top par rapport à body

Relative : left/top décalent par rapport à la position static

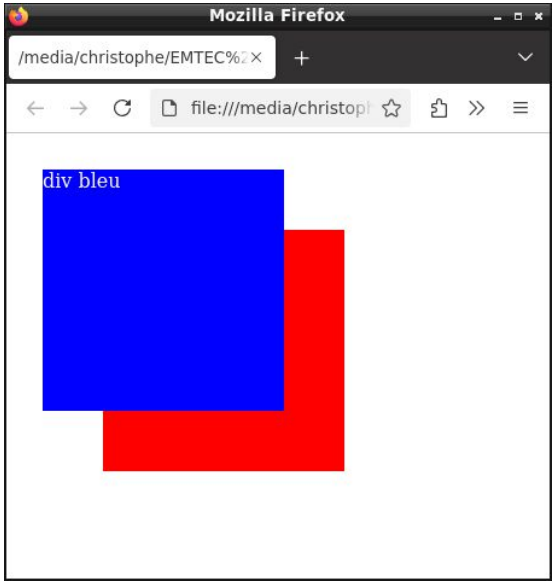
Absolute : positionné avec left/top par rapport au conteneur

Exemples pour le z-index

```
1   <head>
2     <style type="text/css">
3       div{
4           color:white;
5           width:200px;
6           height:200px;
7           position:absolute;
8       }
9       div.bleu {
10          background-color:blue;
11          top:30px;
12          left:30px;
13          z-index:1;
14      }
15      div.red{
16          background-color:red;
17          top:80px;
18          left:80px;
19          z-index:2;
20      }
21    </style>
22    </head>
23  <body>
24    <div class="bleu">div bleu  </div>
25    <div class="red"> div rouge </div>
26    </div>
27  </body>
```



En inversant les valeurs du z-index

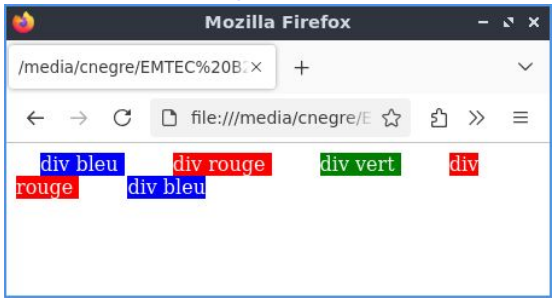


La propriété display

- Les balises un comportement par défaut soit de bloc (body,div,p,...) ou en ligne (span,...) ou d'item de liste (li), ...
- On peut changer ce comportement avec display qui prend les valeurs none, inline, block, list-item, table....

```
1   <head>
2     <style type="text/css">
3       div{
4           color:white;
5           width:70px;
6           height:70px;
7     margin:20px;
8     display:inline;
9       }
10      div.bleu {background-color:blue;}
11      div.red{background-color:red;}
12      div.green{background-color:green;
            }
13    </style>
14    </head>
15  <body>
16    <div class="bleu">div bleu  </div>
17    <div class="red"> div rouge </div>
18    <div class="green"> div vert </div>
19    <div class="red"> div rouge </div>
20    <div class="bleu"> div bleu </div>
21  </body>
```

avec display:inline;

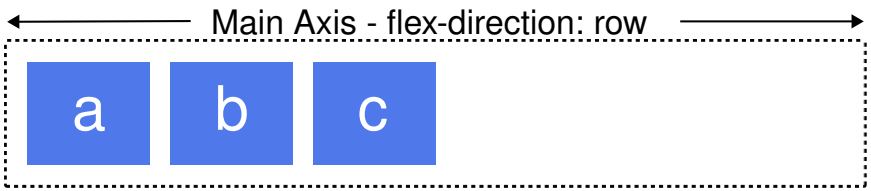


avec display:block;

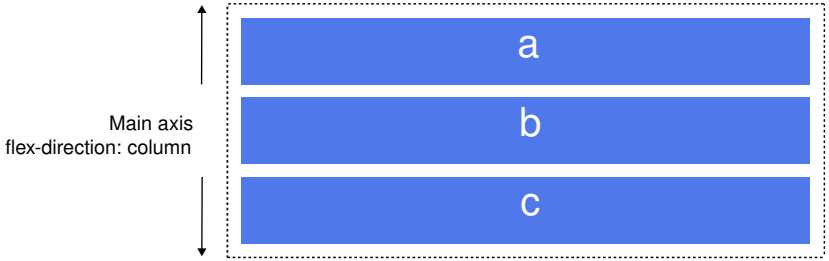
Organisation en boîtes flexibles

Il faut définir display:flex; et choisir la direction:

- flex-direction:row;



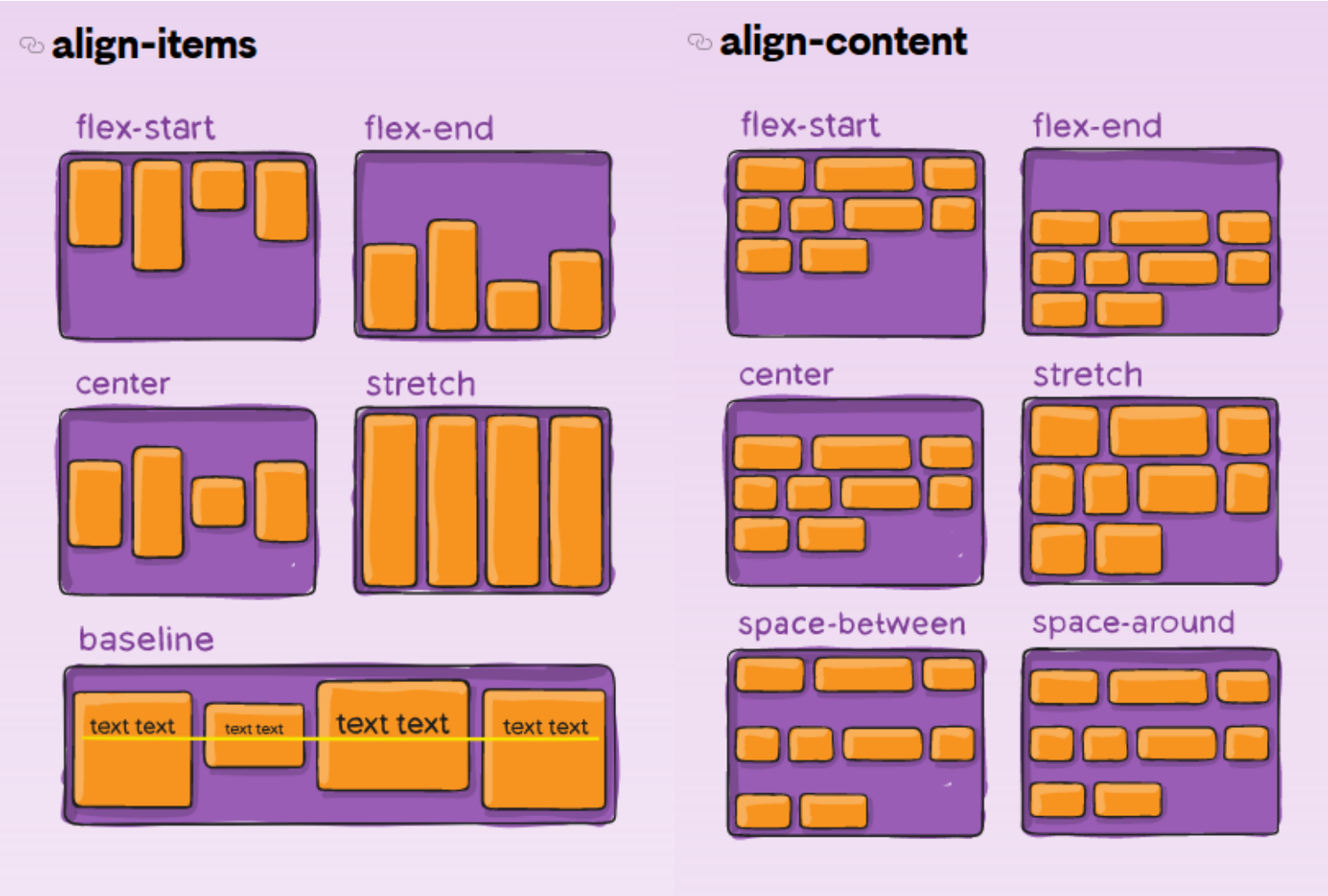
- flex-direction:column;



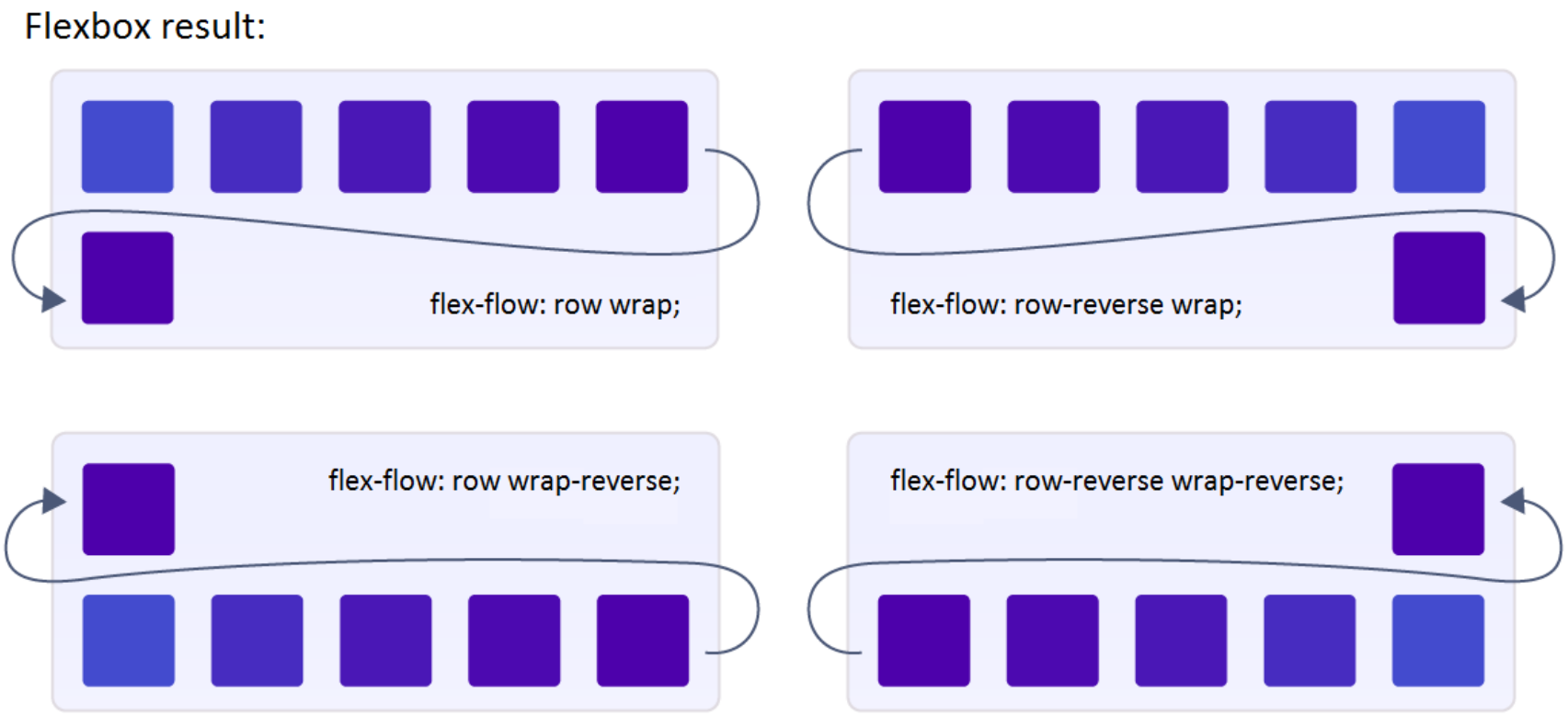
Notons que les valeurs: row-reverse,column-reverse ont le même effet mais dans la direction inverse.

Boites flexibles : centrage horizontal, vertical, etc

Les propriétés `align-items` et `align-content` servent à aligner/espacer les élements.



Boites flexibles : la propriété `wrap`



Position sticky

La position sticky à pour effet de fixer l'élément quand il arrive à distance top du haut de la page.

```
1     <head> <style>
2         body{
3              padding:30px;
4              margin:5px;
5              display:flex;
6              flex-direction:row;
7         }
8         div{
9            margin:5px;
10           width:110px;
11           background-color:red;
12        }
13        div.sticky{
14            height:100px;
15            position:sticky;
16            top:0px; }
17      </style> </head>
18    <body>
19    <div class="sticky"> sticky </div>
20      <div>
21        <div>
22          not sticky 1, not sticky 2,
23          not sticky 3, not sticky 4,
24          not sticky 5, not sticky 6,
25          not sticky 7, not sticky 8,
26          not sticky 9, not sticky 10,
27          not sticky 11,
28        </div>
29    </body>
```

Rendu au chargement de la page

On fait défiler la page : la boite not-sticky remonte et masquée, la boite sticky reste bloquée en haut est visible.

Plan

1. Introduction
2. Syntaxe des feuilles de style
3. Mise en forme du texte
4. Mise en forme de la page
5. Site web reactif (responsive)

Problématique

La consultation d'un site Web peut se faire depuis des appareils très divers :

- Ordinateur de bureau à écran 35 pouces
- Ordinateur portable
- Tablette
- Smartphone
- Version imprimable
- ...

Conception de sites Web réactifs :

La conception de sites Web réactifs (responsive Web design) regroupe différents principes et technologies ayant pour but de faciliter la consultation d'un site Web de façon optimale quel que soit l'appareil utilisé.

Principe technique

La conception de sites Web réactifs repose notamment sur l'utilisation de la règle `@media` en CSS . Utilisation :

- Import de différentes feuilles de style en fonction du navigateur (taille de l'écran, type de système, etc.)
- Redimensionnement de blocs ou d'images
- Personnalisation de parties de la feuille de style

Utilisation des requêtes @media (solution 1)

Exemple d'import conditionnel de feuilles de style dans un fichier HTML :

```
1  <link rel="stylesheet" media="only screen and (min-width:480px)"
2  href = "bureau.css" type="text/css">
3  <link rel="stylesheet" media="handheld, (max-width:480px)"
4  href="smartphone.css" type="text/css">
```

- Si appareil de type screen et taille de l'écran supérieure à 480px : utilisation de bureau.css
- Si appareil de type handheld ou taille de l'écran inférieure à 480px : utilisation de smartphone.css

Utilisation des requêtes @media (solution 2)

Conditions à l'intérieur d'une feuille de style :

```
1  header {
2  width:600px ;
3  background-color:white;
4  }
5  @media only screen and ( max-width:480px ) {
6    header {
7      width:100%;
8      background-color : # D0D0D0 ;
9     }
10 }
```

Utilisation d'expressions booléennes :

- and : "et"
- not : "non"
- , : "ou"
- expressions imbriquées

Exemple

```
1  @media screen and (max-width:480px) and not braille and not projection
```

Bonnes pratiques d'ergonomie

Quelques idées à prendre en considération :

- Menus non affiché par défaut sur terminaux de petite taille (utilisation de javascript ou de pages séparées).
- Eviter l'utilisation du zoom ou du défilement peu pratique pour la navigation.
- Privilégier les tailles relatives en % ou em plutôt qu'absolues (pt,px). Notre que 1em est la taille par défaut du navigateur.
- Privilégier des images vectorielles (svg, pdf) ou plusieurs versions d'images (de différentes tailles).